МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин

ОТЧЁТ

по лабораторной работе № 1

по дисциплине

«Методы решения задач в интеллектуальных системах»

Тема:
Программирование операций обработки данных и знаний
с конвейеризированной обработкой

Вариант:
алгоритм вычисления произведения пары 4-разрядных чисел
умножением с младших разрядов со сдвигом множимого влево

Выполнил: студент гр. 121701
В. А. Герман

Проверил:

Минск, 2026

Содержание

1 Постановка задачи 2

2 Теоретические сведения 2
2.1 Алгоритм умножения с младших разрядов со сдвигом множимого влево . . 2
2.2 Арифметический конвейер 3

3 Программная реализация 3
3.1 Двоичное число фиксированной разрядности 4
3.2 Шаг умножения 4
3.3 Модель конвейера 4

4 Результаты работы программы 5
4.1 Исходные данные 5
4.2 Трассировка конвейера 5
4.3 Результирующий вектор C 7

5 Семейства графиков коэффициента ускорения и эффективности 7

6 Выводы 8

7 Список источников 9

1 Постановка задачи

Тема работы. Программирование операций обработки данных и знаний с конвейеризированной обработкой.

Цель работы. Получить практические навыки разработки модели арифметического конвейера, исследовать зависимости коэффициентов ускорения S и эффективности E от параметров n (число обрабатываемых пар исходных векторов) и r (разрядность операндов, совпадающая с числом этапов конвейера).

Индивидуальный вариант. Алгоритм вычисления произведения пары 4-разрядных чисел умножением с младших разрядов со сдвигом множимого влево.

Требования к модели конвейера.

1. визуальное пошаговое отображение процесса умножения;
2. на каждом этапе конвейера отображаются: данные в двоичном формате, промежуточные результаты, номер (индекс) обрабатываемой пары элементов, время, прошедшее с начала запуска конвейера;
3. перед первым и после последнего этапа отображается не менее трёх пар исходных/результирующих элементов в десятичном формате со своими индексами;
4. двоичные данные отображаются группами по четыре знака, разделяются пробелом (например, 0110 1011);
5. параметрическое задание времени счёта t_i на этапе, числа пар n и разрядности r;
6. время измеряется в условных временных единицах (тактах).

2 Теоретические сведения

2.1 Алгоритм умножения с младших разрядов со сдвигом множимого влево

Пусть заданы два двоичных r-разрядных числа $A = a_{r-1}a_{r-2}\ldots a_0$ — множимое, $B = b_{r-1}b_{r-2}\ldots b_0$ — множитель. Произведение $C = A{\cdot}B$ в общем случае имеет разрядность $2r$ и может быть записано в виде

$$C = \sum_{k=0}^{r-1} b_k \cdot (A \cdot 2^k).$$

Это соотношение непосредственно порождает итеративный алгоритм с младших разрядов множителя. Его особенность в том, что множимое сдвигается влево на каждом шаге, а частичная сумма неподвижна (в отличие от варианта со сдвигом частичной суммы).

Псевдокод одного шага (этапа конвейера):

```
if LSB(F) = 1:          PS := PS + M
M := M << 1
F := F >> 1
```

Здесь M — множимое (хранится в $2r$-разрядном регистре, чтобы сдвиг не приводил к переполнению), F — сдвигаемый вправо множитель (разрядность r), PS — частичная сумма (разрядность $2r$). После r шагов в PS накапливается искомое произведение.

Пример (вручную). Пусть $A = 9 = 1001_2$, $B = 11 = 1011_2$.

Шаг k	M (2r бит)	F	LSB(F)	PS
0	0000 1001	1011	1	0000 0000 $+M$ = 0000 1001
1	0001 0010	0101	1	0000 1001 $+M$ = 0001 1011
2	0010 0100	0010	0	0001 1011
3	0100 1000	0001	1	0001 1011 $+M$ = 0110 0011

Итоговое $PS = 0110\,0011_2 = 99 = 9 \cdot 11$ — верно.

2.2 Арифметический конвейер

Основная идея конвейера заключается в совмещении во времени различных шагов одного и того же алгоритма, применяемого к нескольким парам исходных данных. Пусть

- r — число этапов конвейера (в нашем варианте совпадает с разрядностью множителя: $r = 4$);
- n — число пар исходных операндов;
- t_i — время счёта на i-ом этапе.

Для сбалансированного конвейера ($t_i = t = \text{const}$) имеют место классические соотношения:

$$T_1 = n \cdot r \cdot t, \tag{1}$$

$$T_k = (r + n - 1) \cdot t, \tag{2}$$

$$S = \frac{T_1}{T_k} = \frac{n \cdot r}{r + n - 1}, \tag{3}$$

$$E = \frac{S}{r} = \frac{n}{r + n - 1}. \tag{4}$$

При $n \to \infty$: $S \to r$, $E \to 1$.

3 Программная реализация

В качестве языка реализации выбран Python 3.10+. Проект состоит из следующих модулей:

- binary_number.py — двоичное число фиксированной разрядности (операции $+$, $\ll$, $\gg$, форматированный вывод группами по 4 разряда);
- pipeline.py — общая модель конвейера и конкретный шаг умножения;
- report_html.py — построение интерактивного HTML-отчёта с подробной трассировкой;
- plots.py — построение семейств графиков $S(n)$, $E(n)$, $S(r)$, $E(r)$;
- main.py — точка входа, CLI-параметры (n, r, t_i, входной файл).

3.1 Двоичное число фиксированной разрядности

Класс BinaryNumber хранит биты в списке (от старшего к младшему), поддерживает сложение «в столбик» с переносом, сдвиги влево/вправо и форматированный вывод групп по четыре разряда с разделителем-пробелом (требование 9).

Листинг 1: Фрагмент binary_number.py

```
def shl(self, places: int = 1) -> "BinaryNumber":
    new_bits = self.bits[places:] + [False] * places
    new_bits = new_bits[: self.width]
    return BinaryNumber(width=self.width, bits=new_bits)

def __add__(self, other):
    width = max(self.width, other.width)
    a = BinaryNumber.from_int(width, self.to_int())
    b = BinaryNumber.from_int(width, other.to_int())
    result = [False] * width
    carry = False
    for i in range(width):
        ba = a.bit_lsb(i); bb = b.bit_lsb(i)
        s  = ba ^ bb ^ carry
        carry = (ba and bb) or (ba and carry) or (bb and carry)
        result[width - 1 - i] = s
    return BinaryNumber(width=width, bits=result)
```

3.2 Шаг умножения

Шаг конвейера реализован как преобразование тройки MultiplicationTriple = (M, F, PS).

Листинг 2: Шаг конвейера (pipeline.py)

```
def do_work(self):
    t = self.content
    if t.factor.bit_lsb(0):          # младший бит множителя == 1
        new_ps = t.partial_sum + t.multiplicand
    else:
        new_ps = t.partial_sum
    new_m = t.multiplicand.shl(1)    # сдвиг множимого ВЛЕВО
    new_f = t.factor.shr(1)          # сдвиг множителя вправо (извлекаем LSB)
    self.content = MultiplicationTriple(
        index=t.index, multiplicand=new_m,
        factor=new_f,  partial_sum=new_ps,
    )
```

3.3 Модель конвейера

Конвейер содержит r этапов. На каждом такте: (а) содержимое этапов сдвигается на один этап вперёд; (б) в первый этап загружается очередная пара; (в) каждый этап выполняет do_work; (г) снимается выходной результат с последнего этапа. Время, прошедшее с начала запуска, равно $\text{tick} \cdot t_i$.

Листинг 3: Такт работы конвейера

```
def _do_tick(self, queue, output):
    self.tick += 1
    for i in range(len(self.stages) - 1, 0, -1):
        self.stages[i].content = self.stages[i - 1].content
```

```
    self.stages[0].content = None
    if queue:
        self.stages[0].content = queue.pop(0)
    for st in self.stages:
        st.do_work()
    self.history.append([s.output_snapshot for s in self.stages])
    self.time_history.append(self.tick * self.stage_time)
    last = self.stages[-1].content
    if last is not None:
        output.append(last)
        self.results.append((last.index, last, self.tick * self.stage_time))
        self.stages[-1].content = None
```

4 Результаты работы программы

4.1 Исходные данные

Файл input.json содержит список пар целых чисел (десятичное представление). Для демонстрации взяты три пары:

i	A	B	Ожидаемое $C = A \cdot B$
1	15	15	225
2	1	7	7
3	9	11	99

4.2 Трассировка конвейера

В таблице 1 приведена пошаговая трассировка работы конвейера ($r = 4$ этапа, $t_i = 1$ у.в.е.). В каждой ячейке указаны: индекс пары i, значение множимого M, множителя F и частичной суммы PS в двоичном формате (группами по 4 разряда) и в десятичном — в скобках. Прочерк «—» означает, что этап на данном такте свободен.

Таблица 1: Пошаговая работа конвейера для трёх пар

Этап/Такт	T1, t=1	T2, t=2	T3, t=3	T4, t=4	T5, t=5	T6, t=6
Этап 1	i=1 M=0001 1110 (30) F=0111 (7) PS=0000 1111 (15)	i=2 M=0000 0010 (2) F=0011 (3) PS=0000 0001 (1)	i=3 M=0001 0010 (18) F=0101 (5) PS=0000 1001 (9)	—	—	—
Этап 2	—	i=1 M=0011 1100 (60) F=0011 (3) PS=0010 1101 (45)	i=2 M=0000 0100 (4) F=0001 (1) PS=0000 0011 (3)	i=3 M=0010 0100 (36) F=0010 (2) PS=0001 1011 (27)	—	—
Этап 3	—	—	i=1 M=0111 1000 (120) F=0001 (1) PS=0110 1001 (105)	i=2 M=0000 1000 (8) F=0000 (0) PS=0000 0111 (7)	i=3 M=0100 1000 (72) F=0001 (1) PS=0001 1011 (27)	—
Этап 4	—	—	—	i=1 M=1111 0000 (240) F=0000 (0) PS=1110 0001 (225)	i=2 M=0001 0000 (16) F=0000 (0) PS=0000 0111 (7)	i=3 M=1001 0000 (144) F=0000 (0) PS=0110 0011 (99)

4.3 Результирующий вектор C

После последнего этапа конвейера формируются элементы результирующего вектора с отметкой времени их появления:

i	C (двоичное)	C (десятичное)	t, у.в.е.
1	1110 0001	225	4
2	0000 0111	7	5
3	0110 0011	99	6

Результаты совпадают с ожидаемыми значениями, что подтверждает корректность работы модели конвейера.

5 Семейства графиков коэффициента ускорения и эффективности

Согласно формулам (3) и (4), построены два семейства графиков: при фиксированных r (варьируется n) и при фиксированных n (варьируется r).

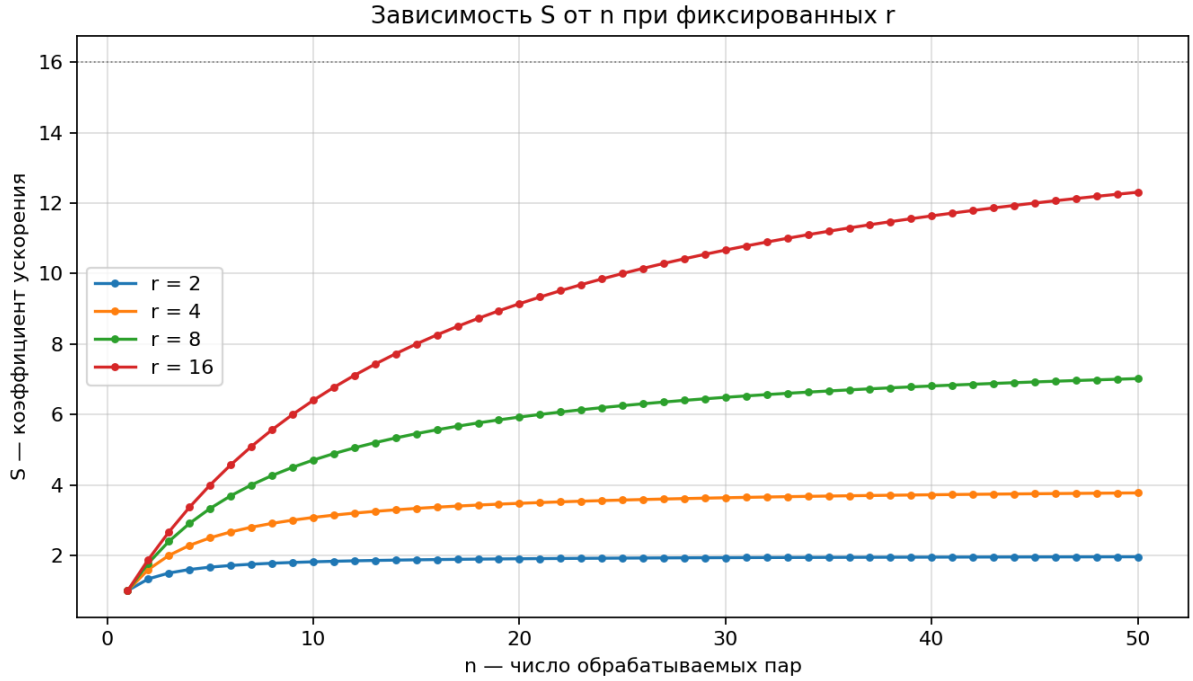

Рис. 1: Семейство $S(n)$ при фиксированных $r \in \{2, 4, 8, 16\}$

Анализ зависимостей.

- При фиксированном r коэффициент ускорения $S(n)$ монотонно возрастает и стремится к значению r; это характерно для конвейера — асимптотическое ускорение равно числу этапов.
- Эффективность $E(n) = S/r$ стремится к единице, причём рост наиболее резкий на малых n, а затем график выходит на «плато». Отсюда следует важный практический вывод: чем больше n, тем лучше окупаются издержки «разгона» конвейера.

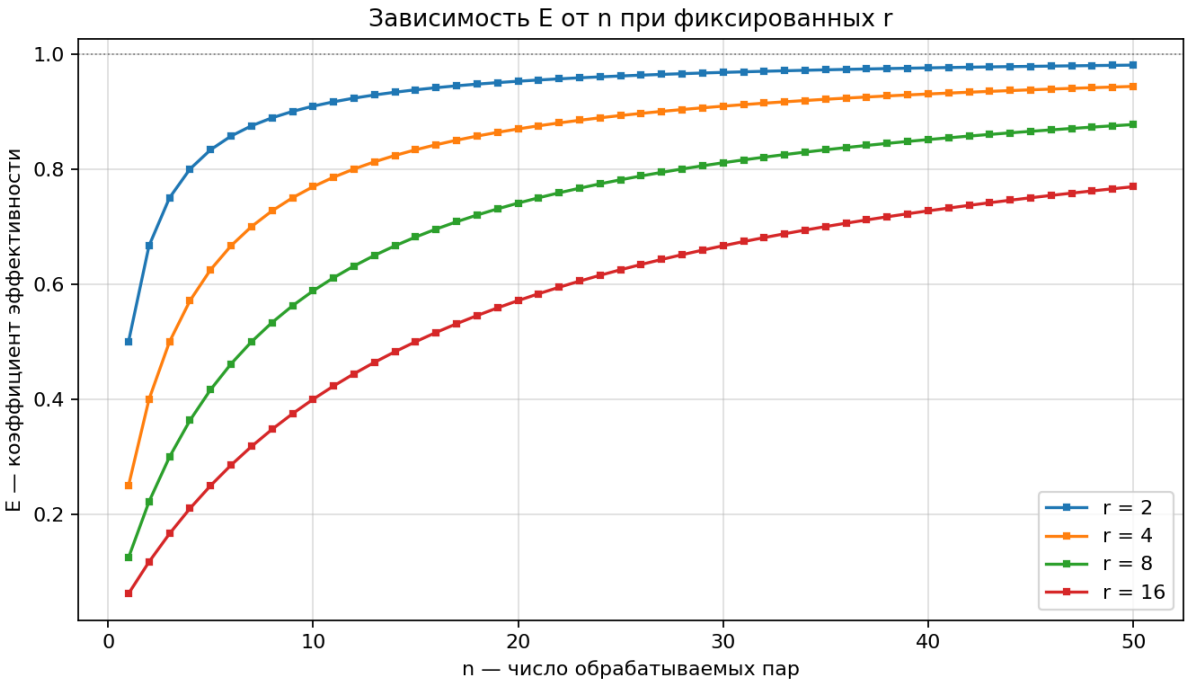

Рис. 2: Семейство $E(n)$ при фиксированных $r \in \{2, 4, 8, 16\}$

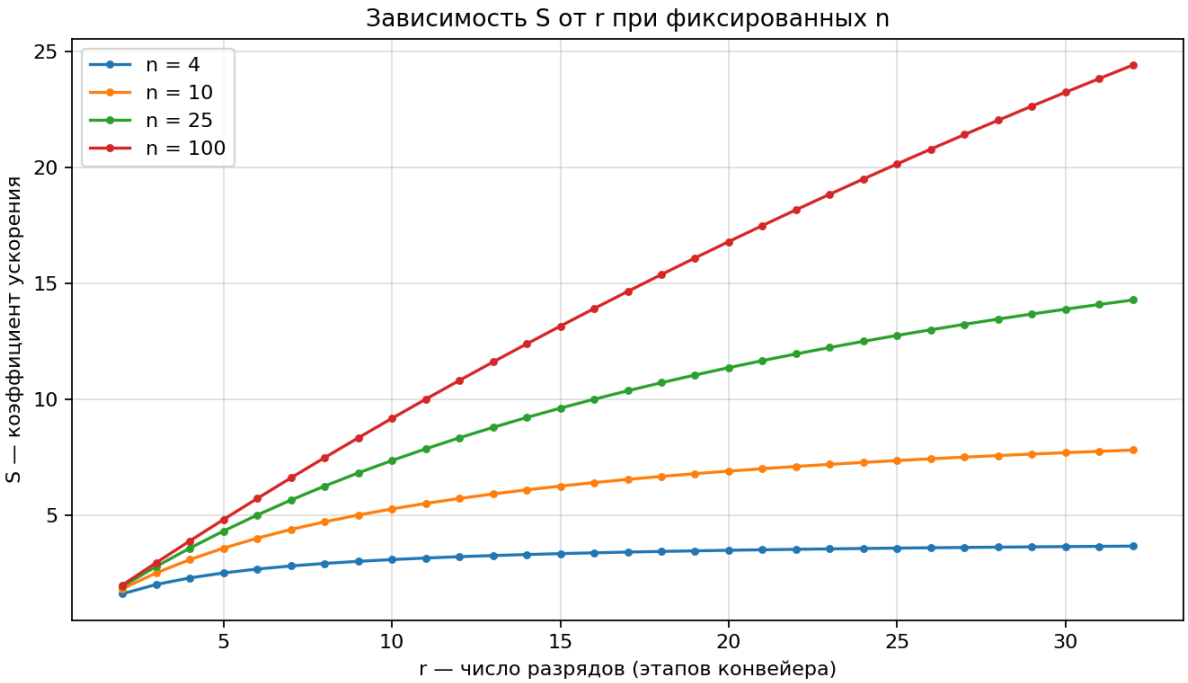

Рис. 3: Семейство $S(r)$ при фиксированных $n \in \{4, 10, 25, 100\}$

- При фиксированном n увеличение r повышает теоретический потолок ускорения, но одновременно снижает эффективность, пока n не увеличится вместе с ним. Это наглядно иллюстрирует компромисс «глубина конвейера — загрузка».

6 Выводы

В ходе лабораторной работы:

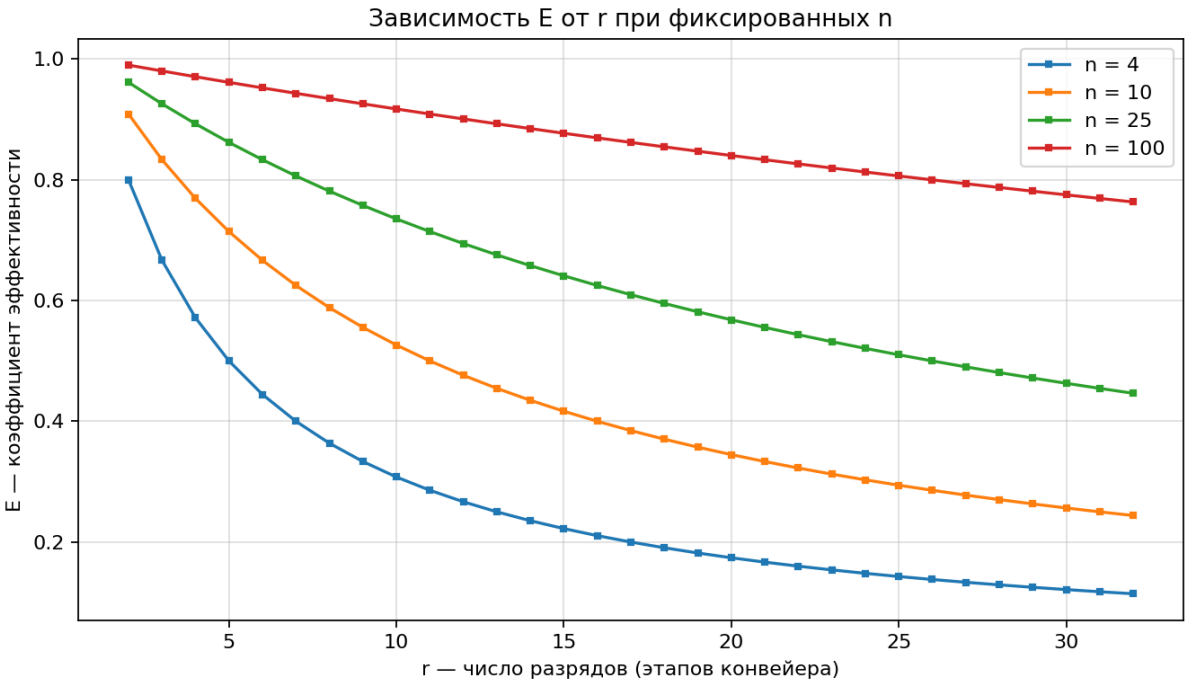

Рис. 4: Семейство $E(r)$ при фиксированных $n \in \{4, 10, 25, 100\}$

- разработана программная модель сбалансированного арифметического конвейера для индивидуального варианта — умножения 4-разрядных чисел с младших разрядов со сдвигом множимого влево;
- обеспечено пошаговое визуальное отображение процесса умножения с индексами пар, двоичными данными (группы по 4 разряда) и временной отметкой в у.в.е.;
- реализовано параметрическое задание n, r, t_i через аргументы командной строки;
- построены и проанализированы семейства графиков коэффициентов ускорения S и эффективности E, подтверждающие классические асимптотики $S \to r$, $E \to 1$ при $n \to \infty$;
- получены практические навыки работы с конвейерными структурами и представлением двоичных чисел фиксированной разрядности.

7 Список источников

1. Качинский, М. В. Арифметические основы электронных вычислительных средств: учебно-методическое пособие / М. В. Качинский, В. Б. Клюс, А. Б. Давыдов. — Минск: БГУИР, 2014. — 64 с.
2. Таненбаум, Э. Архитектура компьютера / Э. Таненбаум. — 6-е изд. — СПб.: Питер, 2022.
3. Хорошевский, В. Г. Архитектура вычислительных систем. — М.: МГТУ им. Баумана, 2008.
4. Репозиторий-пример (работа коллеги, гр. 121701): https://github.com/rastsislaux/bsuir/tree/main/semester-6/MRZvIS/LW1